INFORMATICA · CLASSE QUINTA

Subquery con EXISTS e NOT EXISTS

Subquery correlate, NOT EXISTS e i valori NULL, subquery in HAVING, doppio NOT EXISTS (divisione). Esempi sul database Chinook

Come leggere questi appunti. I riquadri blu contengono le **definizioni** da sapere, quelli verdi gli **esempi**, quelli gialli i punti in cui è **facile sbagliare**. Tutte le query sono state eseguite sul database Chinook e i risultati riportati sono quelli reali. In fondo trovi uno schema riassuntivo e le domande di autoverifica con le risposte.

1. Il database di riferimento: Chinook

Chinook è un database di esempio molto usato per esercitarsi con SQL. Rappresenta un **negozio di musica digitale**: ci sono gli artisti e i loro album, i brani in vendita, i clienti, le fatture con cui i clienti hanno comprato i brani e i dipendenti del negozio. I nomi di tabelle e colonne sono in inglese e al singolare (`Customer`, non `Customers`).

Le dimensioni sono abbastanza piccole da poter controllare i risultati a mano, ma abbastanza grandi da rendere gli esempi realistici: **59 clienti**, **412 fatture**, **3503 brani**, **347 album**, **275 artisti**, 25 generi musicali, 5 formati di file e 8 dipendenti.

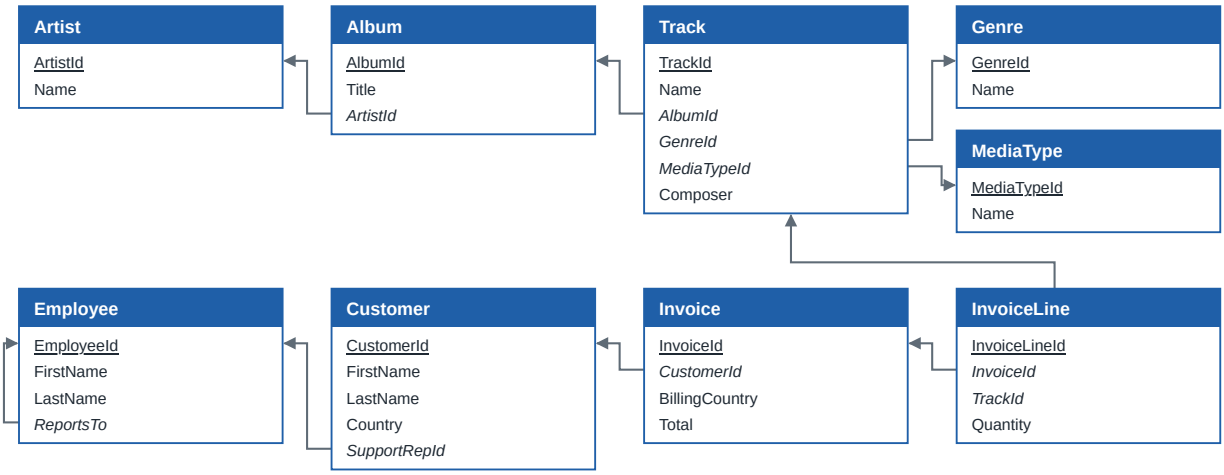


Figura 1 – Le tabelle di Chinook usate nella dispensa (solo le colonne che servono). Chiave primaria sottolineata, chiavi esterne in corsivo; la freccia va dalla chiave esterna alla chiave primaria a cui si riferisce. `ReportsTo` indica il capo del dipendente, cioè un altro dipendente.

I collegamenti da ricordare sono pochi. Un **album** appartiene a un artista; un **brano** (`Track`) appartiene a un album e ha un genere e un formato. Una **fattura** (`Invoice`) è intestata a un cliente ed è composta da **righe** (`InvoiceLine`), ognuna delle quali corrisponde a un brano acquistato. Quindi «il cliente X ha comprato il brano Y» si ricava passando per `Invoice` e `InvoiceLine`.

2. Subquery e annidamento

2.1 Che cos'è una subquery

> **DEFINIZIONE**
> Una **subquery** (o **query annidata**) è un comando `SELECT` scritto, tra parentesi, dentro un'altra query. La query che la contiene si chiama **query esterna**.

In questa dispensa la subquery compare in due punti: nella clausola **WHERE**, dove serve a decidere quali *righe* tenere, e nella clausola **HAVING**, dove serve a decidere quali *gruppi* tenere. Una subquery può a sua volta contenerne un'altra: si parla allora di più **livelli di annidamento**, come vedremo nel paragrafo 6.

2.2 Subquery semplice e subquery correlata

Le subquery si dividono in due tipi, e la differenza è fondamentale per capire EXISTS.

> **DEFINIZIONE**
> Una subquery è **semplice** (non correlata) se non usa nessuna colonna della query esterna: il DBMS la calcola **una volta sola** e ne usa il risultato.
>
> Una subquery è **correlata** se usa una colonna della query esterna: il suo risultato cambia da riga a riga, quindi il DBMS la **rivaluta per ogni riga** della query esterna.

	Subquery semplice	**Subquery correlata**
Usa colonne esterne?	No	Sì (es. `i.CustomerId = c.CustomerId`)
Quante volte viene calcolata	Una volta	Una volta per ogni riga esterna
Si può eseguire da sola?	Sì	No, manca il valore della riga esterna
Esempio in Chinook	`(SELECT AVG(Total) FROM Invoice)`	tutte le subquery con EXISTS di questa dispensa

3. L'operatore EXISTS

> **DEFINIZIONE**
> `EXISTS (subquery)` è una condizione che vale **VERO** se la subquery restituisce **almeno una riga** e **FALSO** se non ne restituisce nessuna.

EXISTS non guarda *che cosa* contengono le righe, ma solo *se ci sono*. Per questo nella subquery si scrive per convenzione `SELECT *` (oppure `SELECT 1`): le colonne scelte non cambiano il risultato.

ESEMPIO · CLIENTI CON UNA FATTURA IMPORTANTE

Vogliamo i clienti che hanno almeno una fattura di **20 dollari o più**.

```
SELECT c.FirstName, c.LastName, c.Country
FROM Customer c
WHERE EXISTS (
    SELECT *
    FROM Invoice i
    WHERE i.CustomerId = c.CustomerId   -- correlazione con la query esterna
      AND i.Total >= 20
);
```

Risultato: 4 clienti su 59, cioè Helena Holý (Czech Republic), Richard Cunningham (USA), Ladislav Kovács (Hungary) e Hugh O'Reilly (Ireland).

La riga `i.CustomerId = c.CustomerId` è la **condizione di correlazione**: collega la fattura `i` della subquery al cliente `c` della query esterna. È lei che trasforma la domanda generica «esistono fatture da 20 dollari?» nella domanda giusta: «esistono fatture da 20 dollari *di questo cliente*?».

3.1 Come viene valutata

Il modo più semplice per capire una subquery correlata è immaginare che il DBMS lavori così (il motore reale può ottimizzare, ma il risultato è lo stesso):

1. prende il **primo cliente** della tabella `Customer`;
2. esegue la subquery mettendo al posto di `c.CustomerId` il codice di quel cliente;
3. se la subquery trova almeno una fattura, EXISTS è vero e il cliente entra nel risultato; altrimenti viene scartato;
4. passa al cliente successivo e ripete, fino all'ultimo.

Cliente (CustomerId)	Fatture	Fattura più alta	Righe trovate dalla subquery	EXISTS
Luís Gonçalves (1)	7	13,86 $	0	FALSO → scartato
Leonie Köhler (2)	7	13,86 $	0	FALSO → scartato
František Wichterlová (5)	7	16,86 $	0	FALSO → scartato
Helena Holý (6)	7	25,86 $	1	**VERO → tenuto**

ATTENZIONE · LA CORRELAZIONE DIMENTICATA

Se si dimentica `i.CustomerId = c.CustomerId`, la subquery diventa semplice e chiede solo «esiste in tutto il database una fattura da 20 dollari?». In Chinook ce ne sono 4, quindi EXISTS è vero **per ogni cliente** e la query restituisce **tutti i 59 clienti**. Senza correlazione EXISTS dà sempre tutte le righe oppure nessuna.

4. L'operatore NOT EXISTS

DEFINIZIONE

`NOT EXISTS (subquery)` vale **VERO** quando la subquery restituisce **zero righe**. Si usa per le domande con «mai», «nessuno», «senza».

ESEMPIO · BRANI MAI VENDUTI

```
SELECT t.Name
FROM Track t
WHERE NOT EXISTS (
    SELECT *
    FROM InvoiceLine il
    WHERE il.TrackId = t.TrackId
);
```

Risultato: 1519 brani su 3503 non compaiono in nessuna riga di fattura, cioè non sono mai stati venduti.

ESEMPIO · ARTISTI SENZA ALBUM

```
SELECT a.Name
FROM Artist a
WHERE NOT EXISTS (
    SELECT * FROM Album al WHERE al.ArtistId = a.ArtistId
);
```

Risultato: 71 artisti su 275 non hanno nessun album nel database.

4.1 Le righe senza corrispondenze vengono incluse

NOT EXISTS è vero ogni volta che la subquery è vuota, anche quando è vuota per il motivo «sbagliato». Prendiamo la domanda «artisti che non hanno **nessun album dal vivo**» (titolo che contiene `Live`):

```
SELECT a.Name
FROM Artist a
WHERE NOT EXISTS (
    SELECT * FROM Album al
    WHERE al.ArtistId = a.ArtistId
      AND al.Title LIKE '%Live%'
);
```

Il risultato è di **264 artisti**, ma 71 di questi non hanno proprio nessun album: per loro la subquery è vuota perché non ci sono album, non perché gli album non sono dal vivo.

> **ATTENZIONE**
> Se si vogliono solo gli artisti che *hanno* album, ma nessuno dal vivo, bisogna aggiungere un EXISTS:
>
> ```
> AND EXISTS (SELECT * FROM Album al2 WHERE al2.ArtistId = a.ArtistId)
> ```
>
> Con questa condizione in più il risultato scende a **193 artisti** (264 − 71).

4.2 NOT EXISTS e NOT IN: il problema dei NULL

Spesso NOT EXISTS si può riscrivere con `NOT IN`, ma le due forme **non sono sempre equivalenti**. La differenza compare quando la subquery restituisce un valore **NULL**.

In Chinook la colonna `Employee.ReportsTo` contiene il capo di ogni dipendente. Il direttore generale, Andrew Adams, non ha un capo, quindi per lui `ReportsTo` è **NULL**. Cerchiamo i dipendenti che **non sono capi di nessuno**:

Con NOT IN	Con NOT EXISTS

`SELECT e.LastName`
`FROM Employee e`
`WHERE e.EmployeeId NOT IN (`
`  SELECT ReportsTo`
`  FROM Employee`
`);`
	`SELECT e.LastName`
	`FROM Employee e`
	`WHERE NOT EXISTS (`
	`  SELECT * FROM Employee s`
	`  WHERE s.ReportsTo = e.EmployeeId`
	`);`

0 righe	**5 righe**: Jane Peacock, Margaret Park, Steve Johnson, Robert King, Laura Callahan

Il motivo sta nel modo in cui SQL tratta il NULL. `x NOT IN (2, 1, NULL, …)` equivale a `x <> 2 AND x <> 1 AND x <> NULL …`, e il confronto `x <> NULL` non è né vero né falso ma **sconosciuto**. Una condizione in AND con un «sconosciuto» non può mai diventare vera, quindi nessuna riga passa il filtro. NOT EXISTS invece controlla solo se la subquery trova righe, e una riga con NULL semplicemente non soddisfa `s.ReportsTo = e.EmployeeId`.

> **ATTENZIONE**
> Basta **un solo NULL** nel risultato della subquery perché `NOT IN` restituisca zero righe, senza nessun messaggio di errore. Quando la colonna può contenere NULL, usa **NOT EXISTS**.

5. Subquery nella clausola HAVING

Le due clausole di filtro lavorano in momenti diversi. **WHERE** agisce **prima** del raggruppamento e decide quali righe entrano nei gruppi. **HAVING** agisce **dopo** il `GROUP BY` e decide quali **gruppi** tenere. Una subquery in HAVING serve quindi quando la condizione riguarda il gruppo intero.

5.1 Confronto con un valore calcolato

ESEMPIO · PAESI SOPRA LA MEDIA

Paesi in cui l'importo medio delle fatture supera la media generale del negozio.

```
SELECT BillingCountry, AVG(Total) AS media, COUNT(*) AS fatture
FROM Invoice
GROUP BY BillingCountry
HAVING AVG(Total) > (SELECT AVG(Total) FROM Invoice);
```

Risultato: la media generale è 5,65 $ e la superano 10 paesi; il primo è il Cile (6,66 $), l'ultimo la Norvegia (5,66 $).

Questa subquery è **semplice**: viene calcolata una volta e restituisce un solo numero, con cui si confronta la media di ogni gruppo.

5.2 EXISTS e NOT EXISTS dentro HAVING

In HAVING si può usare anche una subquery **correlata** con EXISTS o NOT EXISTS. La correlazione però non può riferirsi a una singola riga, perché dopo il raggruppamento le singole righe non ci sono più: c'è solo il gruppo.

REGOLA

In HAVING una subquery può essere correlata **solo con le colonne del GROUP BY** (o con funzioni di aggregazione), perché sono gli unici valori che hanno un significato per il gruppo intero.

ESEMPIO · ALBUM DI CUI NON SI È MAI VENDUTO NIENTE

```
SELECT t.AlbumId, COUNT(*) AS brani
FROM Track t
GROUP BY t.AlbumId
HAVING NOT EXISTS (
    SELECT *
    FROM Track t2
    JOIN InvoiceLine il ON il.TrackId = t2.TrackId
    WHERE t2.AlbumId = t.AlbumId     -- correlazione sulla colonna del GROUP BY
);
```

Risultato: 43 album, per esempio «Quiet Songs» e «Realize»: nessuno dei loro brani compare in una fattura.

Il gruppo è l'album. La subquery cerca, tra tutti i brani di quell'album (`t2.AlbumId = t.AlbumId`), almeno uno venduto. Se non ne trova nessuno, NOT EXISTS è vero e l'album resta. Usare `t.TrackId` nella correlazione sarebbe un errore: nel gruppo ci sono molti brani e non si saprebbe quale scegliere.

5.3 WHERE o HAVING? Lo stesso NOT EXISTS dà risultati diversi

Proviamo a spostare il controllo nel WHERE, correlato sul singolo brano:

```
SELECT t.AlbumId, COUNT(*) AS brani_invenduti
FROM Track t
WHERE NOT EXISTS (SELECT * FROM InvoiceLine il WHERE il.TrackId = t.TrackId)
GROUP BY t.AlbumId;
```

Dove sta il NOT EXISTS	Che cosa controlla	Risultato in Chinook
WHERE (correlato sul brano)	Scarta i **singoli brani** venduti; poi raggruppa quelli rimasti. Un album resta se ha **almeno un** brano invenduto.	299 album
HAVING (correlato sull'album)	Scarta l'**intero album** se anche un solo suo brano è stato venduto. Un album resta solo se **tutti** i brani sono invenduti.	43 album

> **ATTENZIONE**
> Prima di scegliere, chiediti se la condizione riguarda **ogni riga** («brani invenduti») o **il gruppo intero** («album di cui non si è venduto niente»). Nel primo caso va nel WHERE, nel secondo nel HAVING.

5.4 Più condizioni nello stesso HAVING

> **ESEMPIO · CONDIZIONE SU AGGREGATO E SU EXISTS**
> Paesi con **più di 3 clienti** e in cui **almeno un cliente** ha una fattura sopra i 20 dollari.

```
SELECT c.Country, COUNT(*) AS clienti
FROM Customer c
GROUP BY c.Country
HAVING COUNT(*) > 3
   AND EXISTS (
       SELECT *
       FROM Customer c2
       JOIN Invoice i ON i.CustomerId = c2.CustomerId
       WHERE c2.Country = c.Country
         AND i.Total > 20
   );
```

Risultato: i paesi con più di 3 clienti sono 5 (Brasile, Canada, Francia, Germania, USA), ma solo gli **USA** (13 clienti) hanno anche una fattura sopra i 20 $.

6. Doppio NOT EXISTS: la divisione

Alcune domande contengono «**tutti**» o «**ogni**»: «clienti che hanno comprato *tutti* i formati», «studenti che hanno sostenuto *ogni* esame». In algebra relazionale questa operazione si chiama **divisione**. SQL non ha un operatore «per ogni», ma lo si ottiene con una **doppia negazione**.

DEFINIZIONE

«X è in relazione con **tutti** gli Y» equivale a «**non esiste** un Y con cui X **non** sia in relazione». In SQL si scrive con due NOT EXISTS annidati.

ESEMPIO · CLIENTI CHE HANNO COMPRATO TUTTI I FORMATI

In Chinook ci sono 5 formati di file (`MediaType`). Cerchiamo i clienti che hanno comprato almeno un brano di **ciascun formato**.

```
SELECT c.FirstName, c.LastName
FROM Customer c
WHERE NOT EXISTS (                          -- non esiste...
    SELECT *
    FROM MediaType m
    WHERE NOT EXISTS (                      -- ...un formato per cui non esiste...
        SELECT *
        FROM Invoice i
        JOIN InvoiceLine il ON il.InvoiceId = i.InvoiceId
        JOIN Track t        ON t.TrackId    = il.TrackId
        WHERE i.CustomerId  = c.CustomerId  -- ...un acquisto di QUESTO cliente
          AND t.MediaTypeId = m.MediaTypeId -- ...in QUEL formato
    )
);
```

Risultato: un solo cliente, Bjørn Hansen.

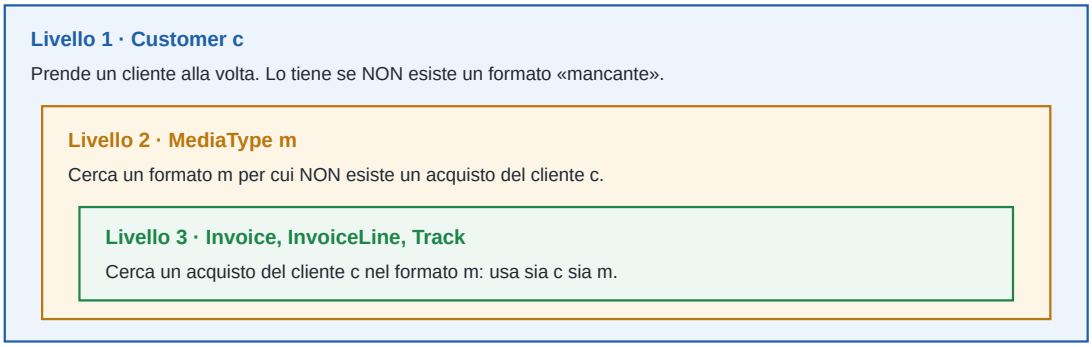


Figura 2 – I tre livelli di annidamento del doppio NOT EXISTS. La query più interna è correlata con entrambe le query che la contengono.

Leggiamo la query dall'interno verso l'esterno. Il **livello 3** risponde a «il cliente c ha comprato qualcosa nel formato m?». Il **livello 2** cerca i formati per cui la risposta è no, cioè i formati «mancanti» per quel cliente. Il **livello 1** tiene il cliente solo se non ci sono formati mancanti: quindi li ha comprati tutti.

ATTENZIONE

La subquery più interna deve essere correlata con **entrambi** i livelli esterni: con il cliente (`i.CustomerId = c.CustomerId`) e con il formato (`t.MediaTypeId = m.MediaTypeId`). Se manca una delle due condizioni, la doppia negazione non funziona più.

6.1 «Tutti» tra alcuni elementi

Lo stesso schema funziona anche su un sottoinsieme: basta filtrare il livello 2. Con `WHERE g.Name IN ('Rock', 'Jazz', 'Blues') AND NOT EXISTS (…)` sulla tabella `Genre` si trovano i clienti che hanno comprato brani di **tutti e tre** questi generi: sono **12**. Se invece si chiedono tutti i 25 generi, il risultato è **vuoto**: nessun cliente li ha comprati tutti, e una query corretta può benissimo restituire zero righe.

6.2 La stessa domanda con HAVING e COUNT

La divisione si può scrivere anche contando: un cliente ha comprato tutti i formati se il numero di **formati diversi** che ha comprato è uguale al numero totale di formati.

```
SELECT c.FirstName, c.LastName
FROM Customer c
JOIN Invoice i      ON i.CustomerId = c.CustomerId
JOIN InvoiceLine il ON il.InvoiceId = i.InvoiceId
JOIN Track t        ON t.TrackId    = il.TrackId
GROUP BY c.CustomerId, c.FirstName, c.LastName
HAVING COUNT(DISTINCT t.MediaTypeId) = (SELECT COUNT(*) FROM MediaType);
```

Anche questa versione restituisce solo Bjørn Hansen. Il `DISTINCT` è indispensabile: senza, si conterebbero tutti i brani acquistati e non i formati diversi.

7. Ripasso

7.1 In sintesi

1. Una **subquery** è una SELECT tra parentesi dentro un'altra query; qui la usiamo in WHERE (filtra righe) e in HAVING (filtra gruppi).
2. Una subquery **semplice** si calcola una volta; una **correlata** usa colonne della query esterna e si rivaluta per ogni riga.
3. **EXISTS** è vero se la subquery restituisce almeno una riga; le colonne non contano, per questo si scrive `SELECT *`.
4. Senza correlazione EXISTS dà lo stesso risultato per tutte le righe (in Chinook: 59 clienti su 59).
5. **NOT EXISTS** è vero se la subquery restituisce zero righe: serve per «mai», «nessuno», «senza».
6. NOT EXISTS include anche le righe senza nessuna corrispondenza; per escluderle si aggiunge un EXISTS.
7. **NOT IN** restituisce zero righe se la subquery contiene anche un solo NULL; NOT EXISTS non ha questo problema.
8. In HAVING la subquery serve per condizioni sul **gruppo**; la correlazione può usare solo colonne del GROUP BY.
9. Lo stesso NOT EXISTS in WHERE controlla le singole righe, in HAVING l'intero gruppo (299 album contro 43).
10. Le domande con «**tutti**» si risolvono con il **doppio NOT EXISTS** oppure con `COUNT(DISTINCT …)` nel HAVING.

7.2 Domande di autoverifica

Prova a rispondere senza guardare gli appunti, poi controlla nella pagina successiva.

1. Che cosa restituisce EXISTS? Perché nella subquery è indifferente scrivere `SELECT *` o `SELECT 1`?
2. Qual è la differenza tra una subquery semplice e una correlata? Quale delle due si può eseguire da sola?
3. Nella query dei clienti con una fattura da almeno 20 $, perché togliendo `i.CustomerId = c.CustomerId` si ottengono tutti i 59 clienti?
4. Scrivi la query che trova gli artisti di Chinook senza nessun album.
5. Perché la query «artisti senza album dal vivo» comprende anche artisti che non hanno nessun album? Come si escludono?
6. Perché la ricerca dei dipendenti che non sono capi di nessuno restituisce zero righe con NOT IN e cinque con NOT EXISTS?
7. In che momento agiscono WHERE e HAVING? Che cosa filtra ciascuno?
8. Nella query degli album mai venduti, perché la correlazione nel HAVING usa `t.AlbumId` e non `t.TrackId`?
9. Perché lo stesso NOT EXISTS dà 299 album nel WHERE e 43 nel HAVING?
10. Come si legge il doppio NOT EXISTS dei clienti che hanno comprato tutti i formati? Con quali due condizioni è correlata la subquery più interna?
11. Riscrivi la stessa divisione usando GROUP BY e HAVING. Perché serve `DISTINCT`?

7.3 Risposte

1. Restituisce VERO se la subquery trova almeno una riga, FALSO se non ne trova. Conta solo l'esistenza delle righe, non il loro contenuto, quindi le colonne scelte non cambiano il risultato.
2. La semplice non usa colonne esterne e si calcola una volta; la correlata usa colonne della query esterna e si rivaluta per ogni riga. Solo la semplice si può eseguire da sola, perché alla correlata manca il valore della riga esterna.
3. Senza correlazione la subquery chiede solo se in tutto il database esiste una fattura da 20 $. Ce ne sono 4, quindi EXISTS è vero per ogni cliente e passano tutti.
4. `SELECT a.Name FROM Artist a WHERE NOT EXISTS (SELECT * FROM Album al WHERE al.ArtistId = a.ArtistId);` (71 artisti).
5. Per un artista senza album la subquery è vuota, quindi NOT EXISTS è vero anche se il motivo è l'assenza di album. Si escludono aggiungendo `AND EXISTS (SELECT * FROM Album al2 WHERE al2.ArtistId = a.ArtistId)`: da 264 si scende a 193.
6. La colonna `ReportsTo` contiene un NULL (il direttore generale). Con NOT IN il confronto `x <> NULL` è sconosciuto e nessuna riga può soddisfare la condizione. NOT EXISTS verifica solo se ci sono righe con `s.ReportsTo = e.EmployeeId`, e il NULL non dà fastidio.
7. WHERE agisce prima del GROUP BY e filtra le singole righe; HAVING agisce dopo e filtra i gruppi.
8. Il gruppo è l'album: dopo il raggruppamento esiste un solo valore di `AlbumId` per gruppo, mentre i `TrackId` sono molti. In HAVING si può correlare solo sulle colonne del GROUP BY.
9. Nel WHERE si scartano i singoli brani venduti, quindi resta ogni album con almeno un brano invenduto (299). Nel HAVING si scarta l'album intero se ha anche un solo brano venduto, quindi restano solo gli album del tutto invenduti (43).
10. «Non esiste un formato per cui non esiste un acquisto di questo cliente», cioè il cliente ha comprato tutti i formati. La subquery interna è correlata con il cliente (`i.CustomerId = c.CustomerId`) e con il formato (`t.MediaTypeId = m.MediaTypeId`).
11. Si uniscono Customer, Invoice, InvoiceLine e Track, si raggruppa per cliente e si scrive `HAVING COUNT(DISTINCT t.MediaTypeId) = (SELECT COUNT(*) FROM MediaType)`. Senza DISTINCT si conterebbero tutti i brani acquistati invece dei formati diversi.